



Tecnologías Web

- 
- > Introducción al protocolo HTTP
 - > Principales componentes de HTTP
 - > Estado y Sesiones HTTP
 - > Funcionalidad Web
 - > Esquemas de codificación
 - > Burp Suite
 - > Otras Herramientas



> Introducción al protocolo HTTP

- > Principales componentes de HTTP
- > Estado y Sesiones HTTP
- > Funcionalidad Web
- > Esquemas de codificación
- > Burp Suite
- > Otras Herramientas

Qué es HTTP?

- > **HyperText Transfer Protocol** es el protocolo que nos permite realizar peticiones de datos y recursos, como por ejemplo documentos HTML.
- > **Es un protocolo cliente-servidor**: es decir, que las peticiones de datos son iniciadas por quien recibirá los datos (el cliente), generalmente un navegador Web.
- > Una página web completa, resulta de la unión de distintos sub-documentos recibidos, como por ejemplo:
 - un documento que especifique el estilo de maquetación de la página web (CSS)
 - el texto y las imágenes, vídeos,
 - scripts, etc...

Qué es HTTP?

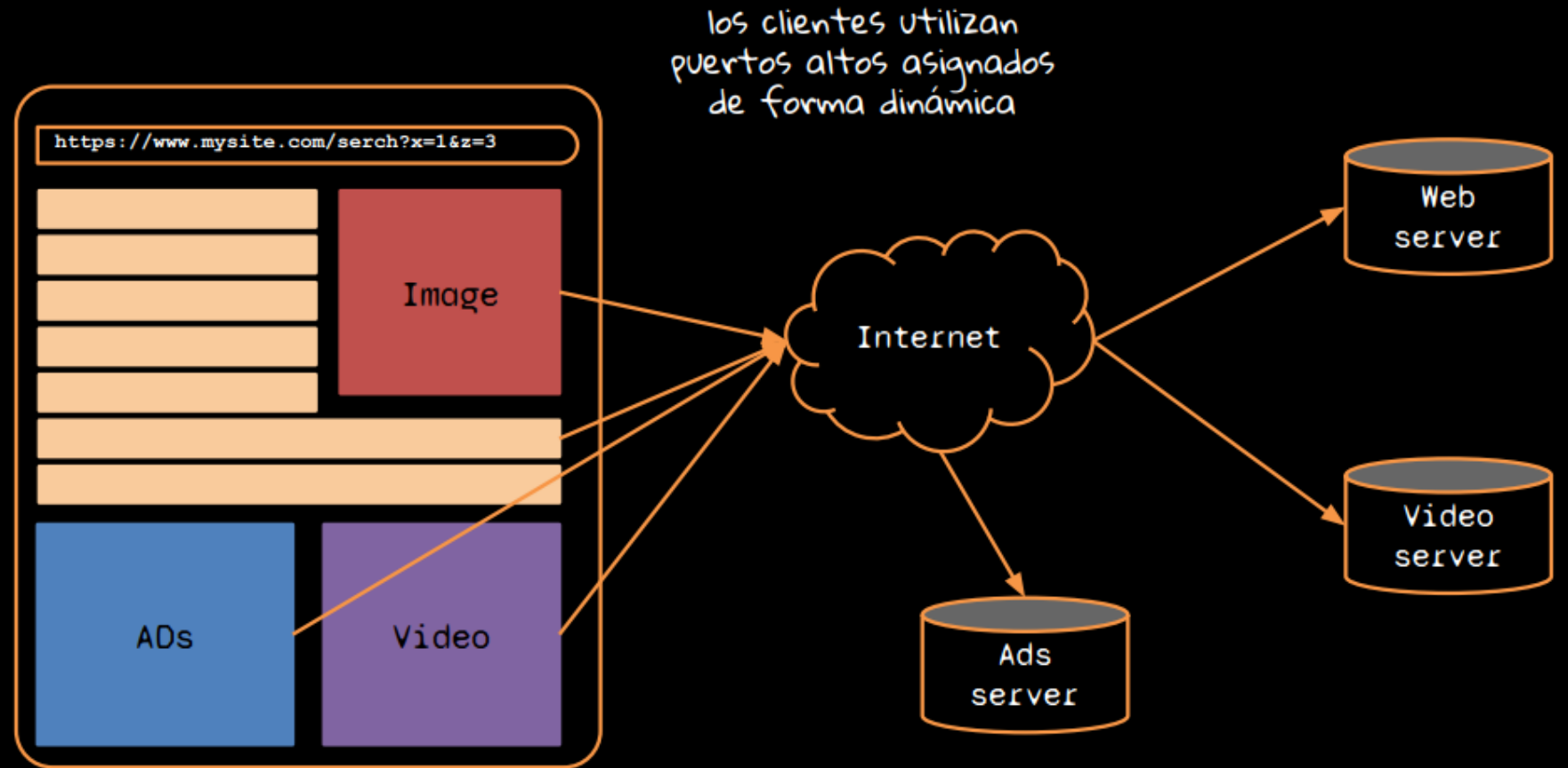
- > Diseñado a principios de la década de 1990 , HTTP es un protocolo extensible que ha ido evolucionando con el tiempo.
- > Es un protocolo de la capa de aplicación , y se transmite sobre el protocolo TCP, o el protocolo encriptado TLS, aunque teóricamente podría usarse cualquier otro protocolo fiable.
- > Gracias a que es un protocolo extensible , se usa no sólo para transmitir documentos de hipertexto (HTML), si no que además, se pueden transmitir imágenes o vídeos, enviar datos o contenido a los servidores, como en el caso de los formularios de datos.
- > HTTP puede incluso ser utilizado para transmitir partes de documentos , y actualizar páginas Web en el acto (Ajax).

INTRODUCCIÓN AL PROTOCOLO HTTP

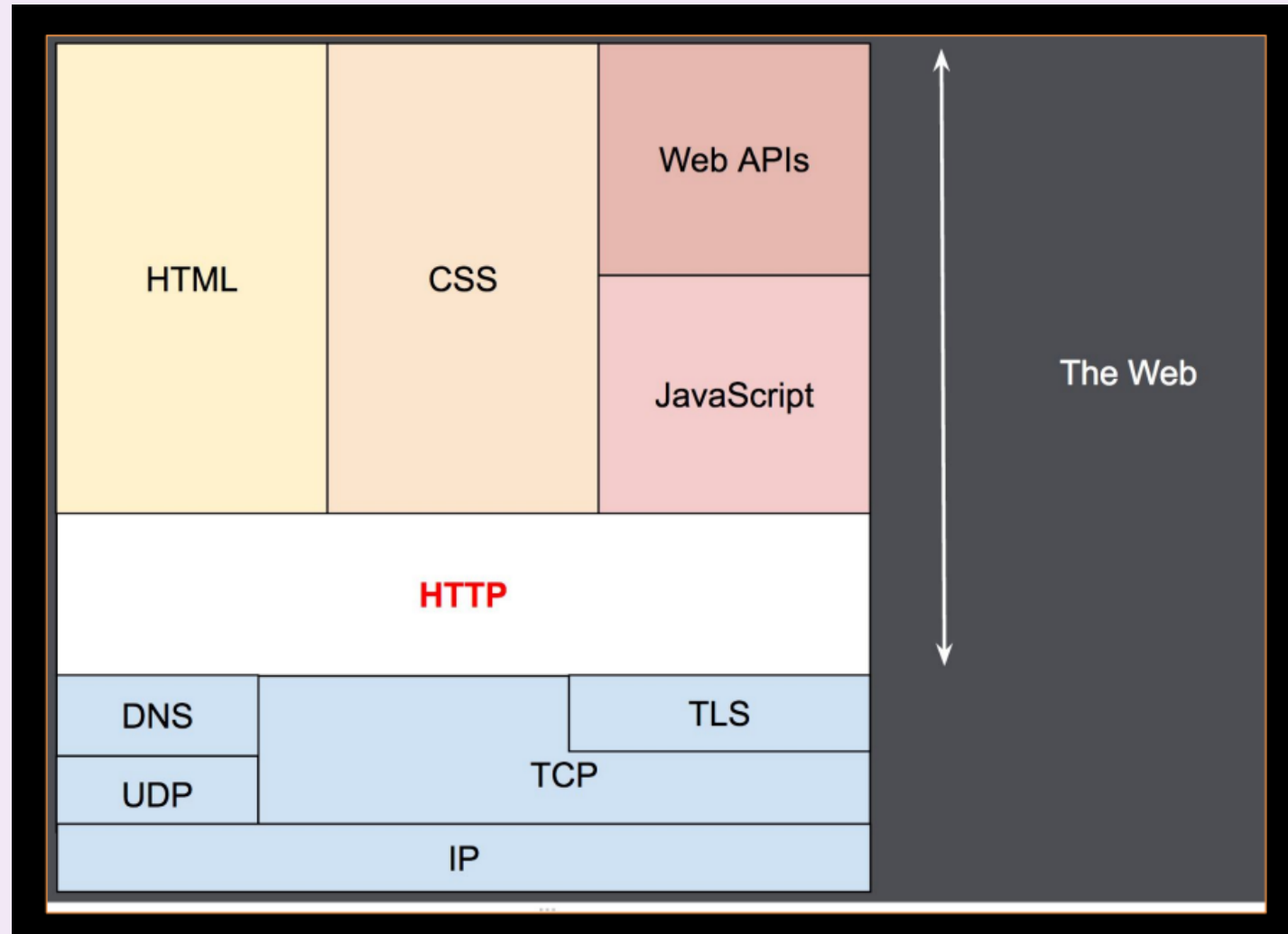
Cientes y servidores se comunican intercambiando mensajes individuales.

Los mensajes que envía el cliente se llaman **peticiones (requests)**

Los mensajes enviados por el servidor se llaman **respuestas (responses)**.

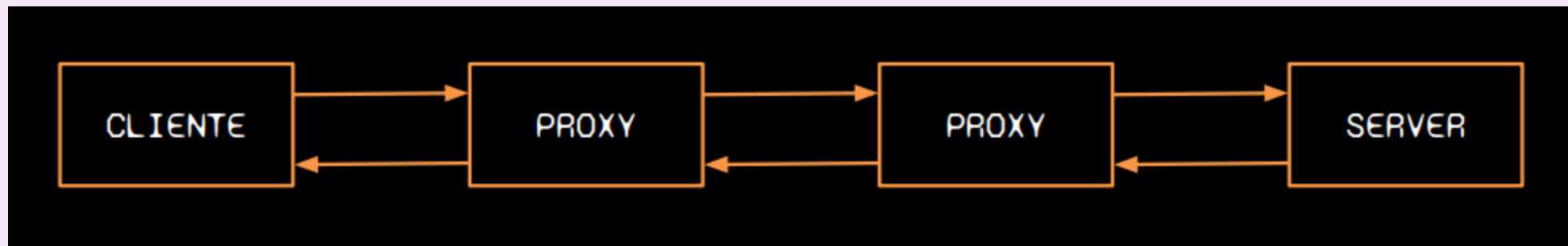


INTRODUCCIÓN AL PROTOCOLO HTTP



Arquitectura de los sistemas basados en HTTP

- > HTTP es un **protocolo basado cliente-servidor**: las peticiones son enviadas por una entidad: el agente del usuario (o un proxy a petición de uno).
- > La mayoría de las veces el agente del usuario (cliente) es un navegador Web, pero podría ser cualquier otro programa.
- > Cada petición individual se envía a un servidor, el cual la gestiona y responde .
- > Entre cada petición y respuesta , hay varios intermediarios , normalmente denominados proxies , los cuales realizan distintas funciones. Ej: gateways o caches.



El cliente

- > El cliente es cualquier herramienta que actúe en representación del usuario. Ej: un navegador Web, curl, wget, etc.
- > El navegador es siempre el que inicia una comunicación (request).
- > El servidor nunca la comienza (hay algunos mecanismos que permiten esto, pero no son muy habituales).
- > Para poder mostrar una página Web, el navegador envía una petición de documento HTML al servidor, procesa este documento, y envía más peticiones para solicitar scripts, hojas de estilo (CSS), y otros datos que necesite (normalmente vídeos y/o imágenes).
- > El navegador une todos estos documentos y datos y compone el resultado final: la página Web.

El servidor web

- > Al otro lado del canal de comunicación, está el servidor, el cual **sirve los datos que ha pedido el cliente**.
- > Físicamente puede estar formado por varios elementos que se reparten la carga de peticiones, (load balancing), u otros programas, que gestionan servicios tales como: caché, bases de datos, servidores de correo electrónico, etc.
- > Un servidor **no tiene que ser necesariamente un único equipo físico**, aunque sí que varios servidores pueden estar funcionando en un único computador.

Proxies

- > Son dispositivos que **gestionan los mensajes HTTP**.
- > Pueden ser transparentes, o no (modificando las peticiones que pasan por ellos), y realizan varias funciones:
 - **caching** (la caché puede ser pública o privada, como la caché de un navegador)
 - **filtrado** (como un anti-virus, control parental, filtrado de contenidos, etc.)
 - **balanceo de carga de peticiones** (para permitir a varios servidores responder a la carga total de peticiones que reciben)
 - **autenticación** (para el control al acceso de recursos y datos)
 - **registro de eventos** (para tener un histórico de los eventos que se producen)

HTTP es sencillo

- > HTTP es un protocolo con un **diseño simple** .
- > Está pensado y desarrollado para ser leído y fácilmente interpretado por las personas.
- > **Es fácil depurar errores** en HTTP.
- > Esta simplicidad reduce la curva de aprendizaje para las personas que recién empiezan a trabajar con HTTP.

HTTP es Extensible

- > Presentadas en la versión HTTP/1.0, las cabeceras de HTTP , han hecho que este protocolo sea fácil de ampliar y de experimentar con él.
- > Desarrollar nuevas funcionalidades es tan sencillo como hacer que un cliente y su servidor comprendan la misma semántica sobre las cabeceras de HTTP.



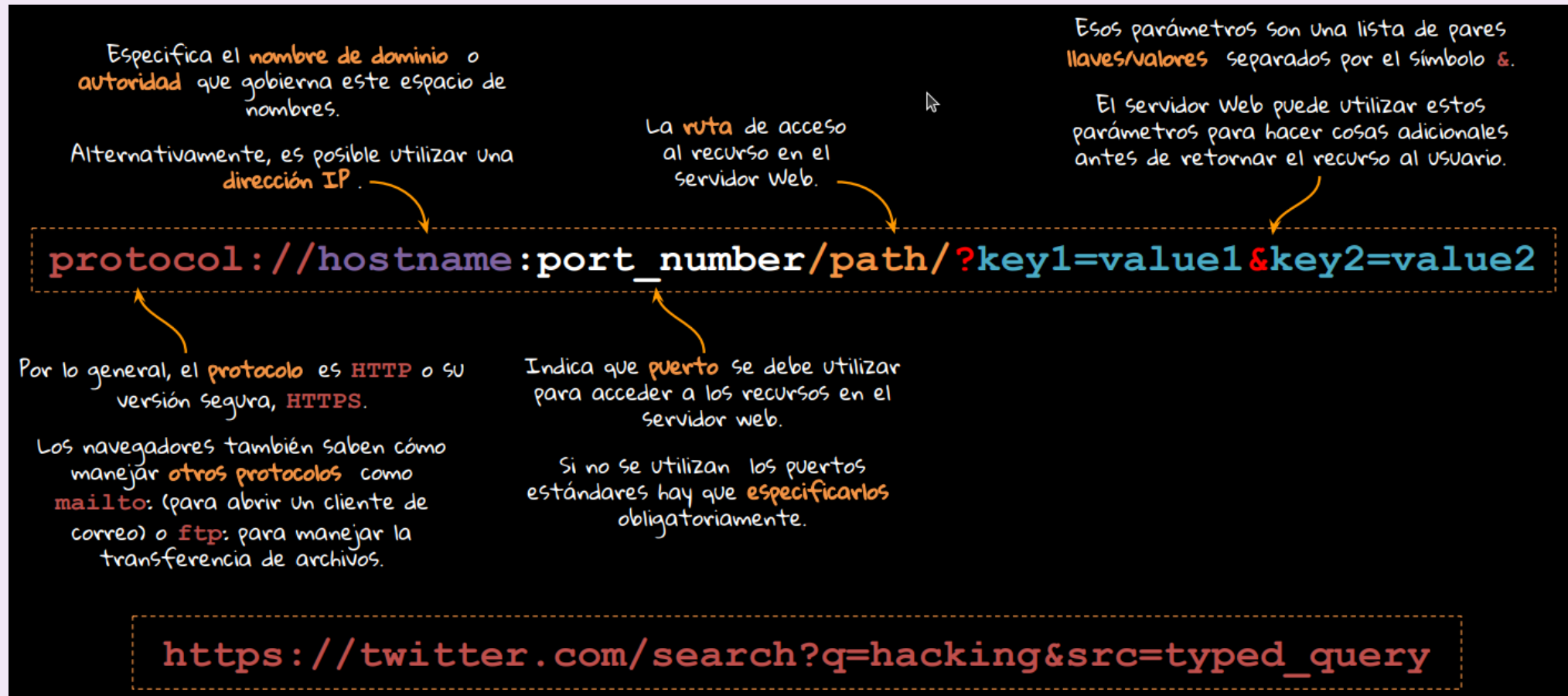
HTTP es Stateless

- > HTTP es un **protocolo sin estado** : no guarda ningún dato entre dos peticiones en la misma sesión.
- > Esto es un problema en caso de que los usuarios requieran interactuar con determinadas páginas Web de forma ordenada y coherente, como por ejemplo, para el uso de carritos de compras.
- > Esto se logra con el uso de cookies , las cuales permiten guardar datos con respecto a una sesión.

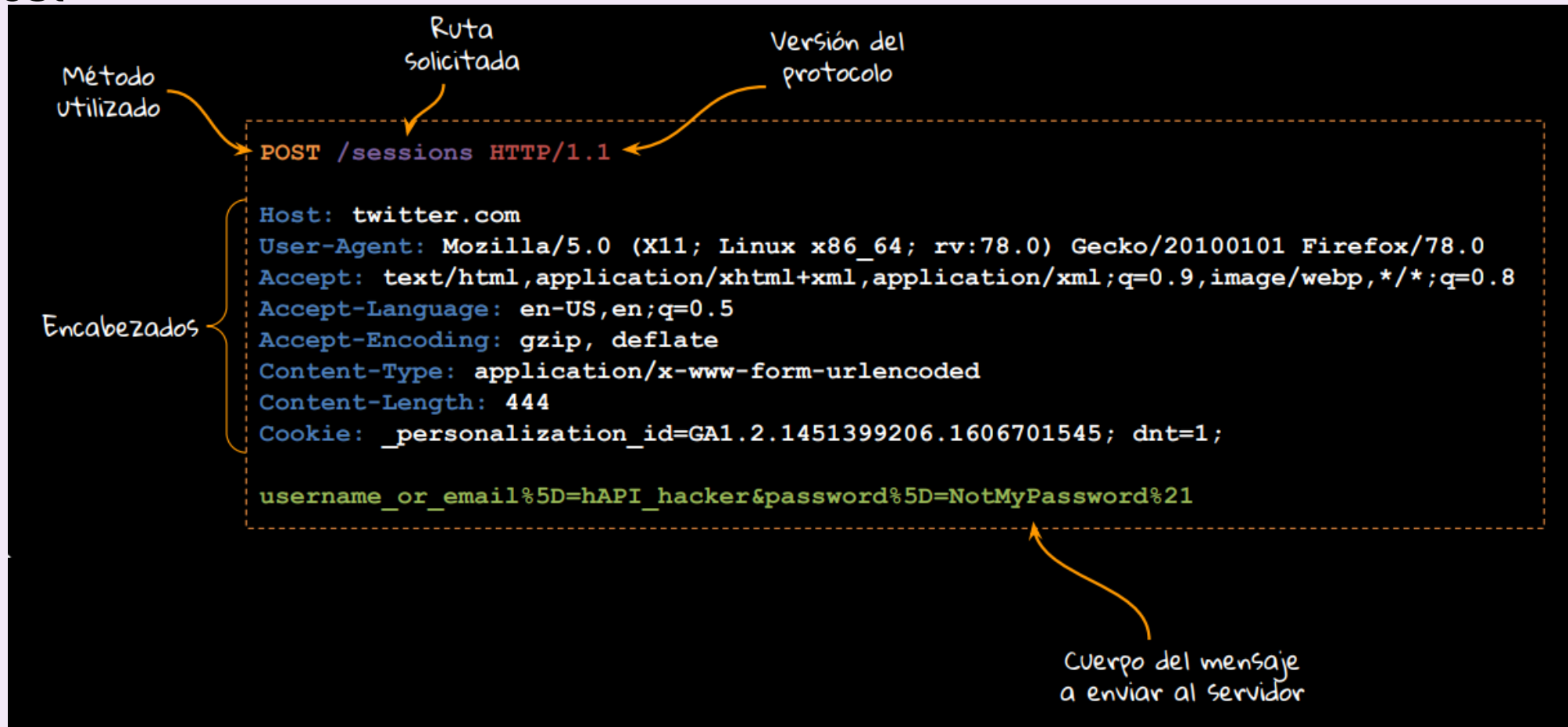


- > Introducción al protocolo HTTP
- > Principales componentes de HTTP
- > Estado y Sesiones HTTP
- > Funcionalidad Web
- > Esquemas de codificación
- > Burp Suite
- > Otras Herramientas

Url



Request



Response

The diagram illustrates the components of an HTTP response. It features a black background with a white dashed box containing the response text. Handwritten orange arrows point from labels to specific parts of the response:

- Versión del protocolo** points to `HTTP/1.1`.
- Código de estado** points to `302`.
- Mensaje del estado** points to `Found`.
- Encabezados** points to the header section, which includes:
 - `content-security-policy: default-src 'none'; connect-src 'self'`
 - `location: https://twitter.com/`
 - `pragma: no-cache`
 - `server: tsa_a`
 - `set-cookie: auth_token=8ff3f2424f8ac1c4ec635b4adb52cddf28ec18b8; Max-Age=157680000; Expires=Mon, 01 Dec 2025 16:42:40 GMT; Path=/; Domain=.twitter.com; Secure; HTTPOnly; SameSite=None`
- Cuerpo del mensaje a enviar al servidor** points to the body section, which contains:

```
<html><body>You are being <a href="https://twitter.com/">redirected</a>.</body></html>
```

Los métodos

> Los métodos indican la acción que se desea realizar.

Método	Descripción
GET	Recopilar recursos del servidor web (documento html, datos de usuario, videos, etc). Si la solicitud tiene éxito, el servidor proporcionará el recurso; caso contrario, proporcionará una respuesta explicando por qué no pudo obtener el recurso solicitado.
POST	Envían datos contenidos en el cuerpo de la solicitud a un servidor web. Si un cliente envía la misma solicitud POST varias veces, el servidor creará varios resultados.
PUT	Indican al servidor web que almacene los datos enviados en la URL solicitada.
HEAD	Pide una respuesta idéntica a la de una petición GET, pero sin el cuerpo de la respuesta.
PATCH	Se utilizan para actualizar parcialmente los recursos con los datos enviados. Es probable que las solicitudes PATCH solo estén disponibles si una respuesta HTTP incluye el encabezado Accept-Patch.
OPTIONS	Es utilizado para describir los métodos habilitados para un recurso determinado.
TRACE	Se utiliza para diagnóstico. El servidor debe retornar en el cuerpo de la respuesta el contenido exacto del request que recibió.
CONNECT	Inician una conexión de red bidireccional. Cuando esté permitido, esta solicitud crearía un túnel proxy entre el navegador y el servidor web.
DELETE	Borra un recurso en específico.

Los headers

- > Las cabeceras HTTP permiten enviar información adicional en un request o response.
- > Están compuestas por su nombre seguido de dos puntos y a continuación su valor.
- > Pueden ser agrupadas de acuerdo a sus contextos :
 - **Generales**: se aplican tanto a request como responses.
 - **De requests**: contienen más información sobre el contenido que va a obtenerse o sobre el cliente.
 - **De responses**: contienen más información sobre el contenido, como su origen o el servidor (nombre, versión, etc.).
 - **De entidad**: contienen más información sobre el cuerpo de la entidad, como el tamaño del contenido o su tipo MIME.

Los Status codes

- > Los códigos de estado de respuesta HTTP indican si se ha completado satisfactoriamente una solicitud HTTP específica.
- > Las respuestas se agrupan en cinco clases :
 - **100s** - son respuestas informativas.
 - **200s** - la solicitud se procesó satisfactoriamente.
 - **300s** - el cliente es redirigido a un recurso diferente.
 - **400s** - el request contiene algún tipo de errores.
 - **500s** - se produjo un error en el servidor al procesar la petición.
- > Los códigos de estado se definen en la sección 10 del [RFC 2616](#). (Pueden obtener las especificaciones actualizadas en el [RFC 7231](#)).

Los Status Code - ejemplos

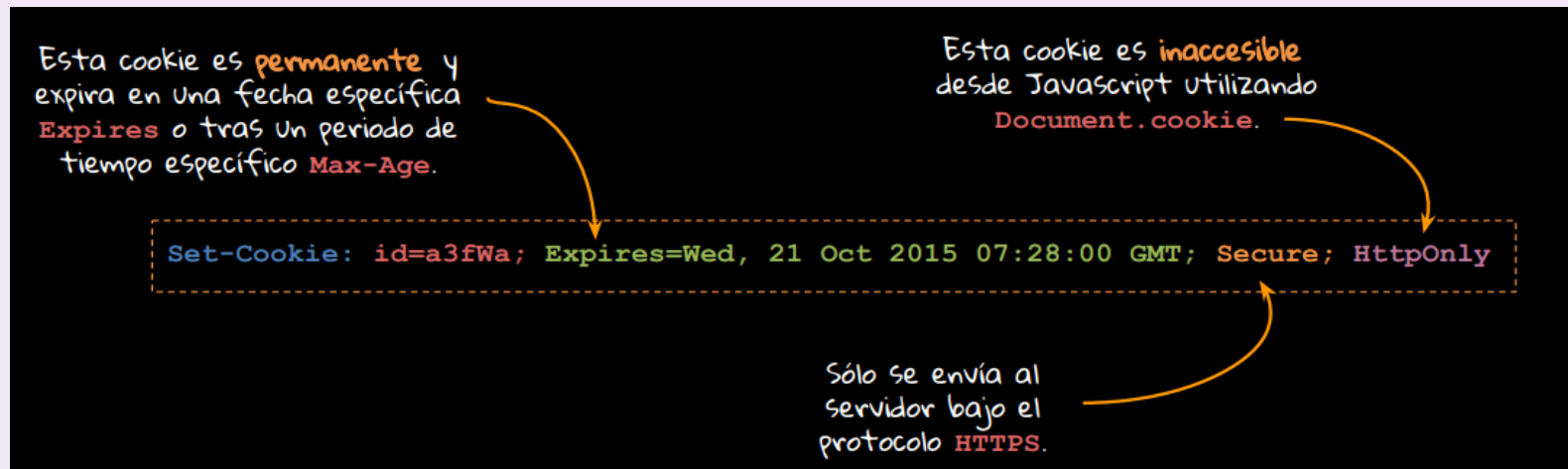
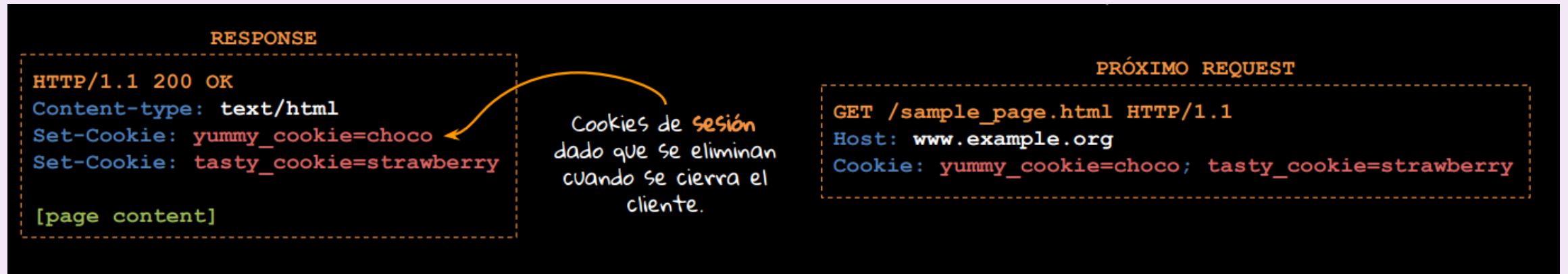
Status Code	Descripción
100 Continue	Es una respuesta provisional que indica que todo hasta ahora está bien y que el cliente debe continuar con la solicitud o ignorarla si ya está terminada.
200 OK	La solicitud ha tenido éxito.
203 Non-Authoritative Information	La solicitud se ha completado con éxito, pero su contenido no se ha obtenido de la fuente originalmente solicitada, sino que se recoge de una copia local o de un tercero.
301 Moved Permanently	La URI del recurso solicitado ha sido cambiado.
302 Found	El recurso de la URI solicitada ha sido cambiado temporalmente.
400 Bad Request	El servidor no pudo interpretar la solicitud dada una sintaxis inválida.
401 Unauthorized	Es necesario autenticar para obtener la respuesta solicitada.
403 Forbidden	El cliente no posee los permisos necesarios para cierto contenido.
404 Not Found	El servidor no pudo encontrar el contenido solicitado.
500 Internal Server Error	El servidor ha encontrado una situación que no sabe cómo manejarla.

Las cookies

- > Una cookie es una **pequeña pieza de datos** que un servidor envía al navegador web del usuario.
- > Para crear una cookie, el servidor envía el **encabezado Set-Cookie** en una respuesta.
- > La cookie debe ser **almacenada por el cliente** y enviada en el encabezado cookie en cada nueva solicitud al mismo servidor.
- > Se utilizan principalmente con tres propósitos:
 - **Gestión de Sesiones:** Inicios de sesión, carritos de compras, puntajes de juegos o cualquier otra cosa que el servidor deba recordar.
 - **Personalización:** Preferencias de usuario, temas y otras configuraciones.
 - **Rastreo:** Guardar y analizar el comportamiento del usuario.



Cookies de session vs permanentes



Alcance de las cookies

- > Las directivas Domain y Path definen a qué URLs deberían enviarse las cookies.
- > **Domain especifica los hosts permitidos.** Si no se especifica, el host siempre es el del Document.location actual excluyendo subdominios .
- > **Path** indica la ruta que debe existir en la URL solicitada para enviar el header.
- > Si Path=/docs estas rutas coincidirán:
 - /docs
 - /docs/Web/
 - /docs/Web/HTTP



Cookies sameSite

- > Permiten a los servidores requerir que una **cookie no sea enviada con solicitudes cross-site**.
- > Se pueden establecer de forma:
 - **Strict**: el navegador sólo enviará cookies al sitio web que la creó.
 - **Lax**: igual que strict pero también se envían cuando un usuario navega hacia el sitio desde un sitio externo, por ejemplo, en un redirect.
 - **None**: las cookies se envían al sitio original y cross-site, pero solo en contexto secure.

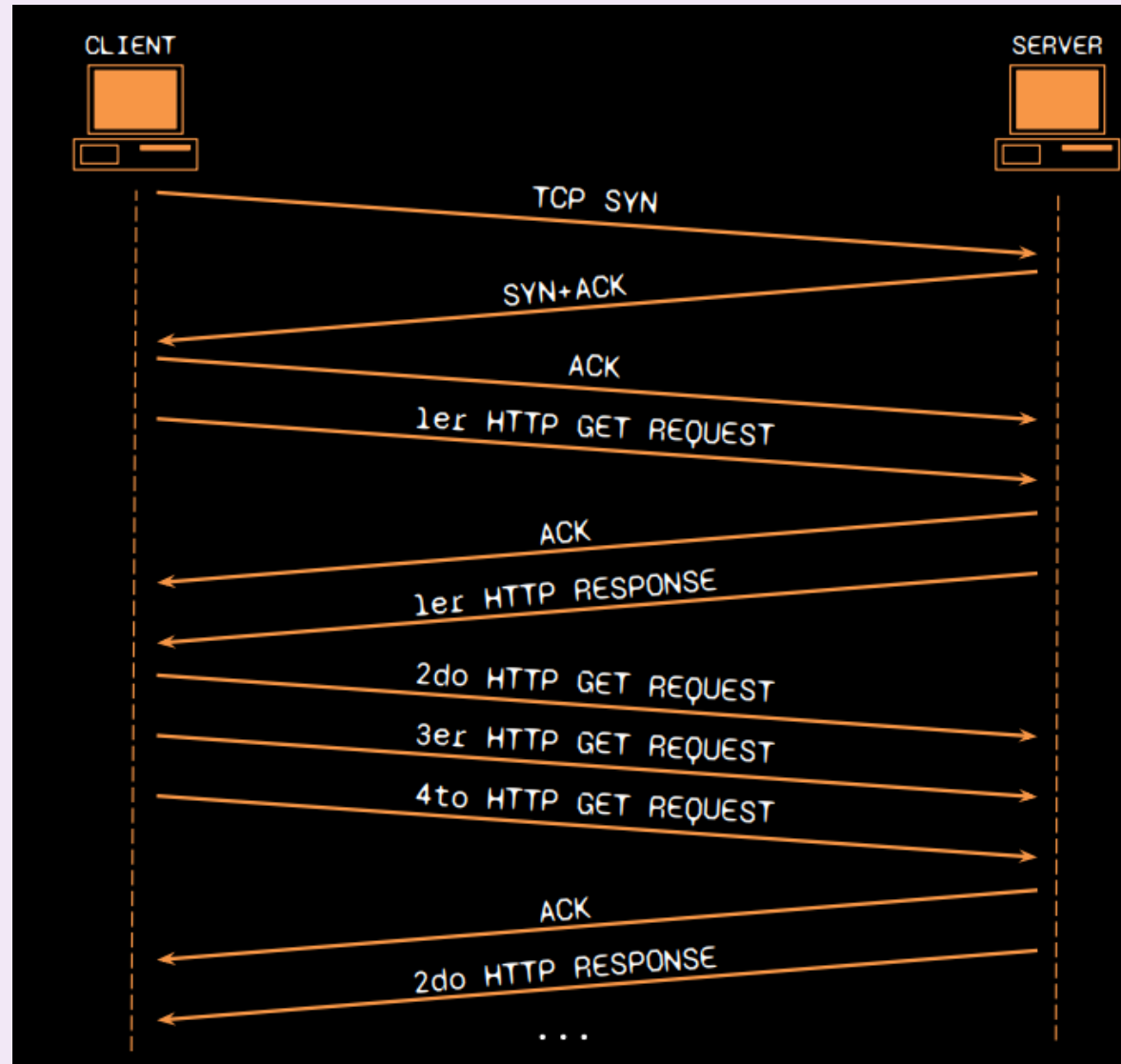
Cookies desde Javascript

- > Se pueden crear nuevas cookies via JavaScript del lado del cliente usando la propiedad `Document.cookie`.
- > Si el flag `HttpOnly` no está establecido, también se puede acceder a las cookies existentes desde JavaScript.

```
document.cookie = "yummy_cookie=choco";  
document.cookie = "tasty_cookie=strawberry";  
console.log(document.cookie);  
// logs "yummy_cookie=choco; tasty_cookie=strawberry"
```

- > Introducción al protocolo HTTP
- > Principales componentes de HTTP
- > Estado y Sesiones HTTP
- > Funcionalidad Web
- > Esquemas de codificación
- > Burp Suite
- > Otras Herramientas

Cómo es el flujo de HTTP?



Stateful vs Stateless

- > El protocolo HTTP es stateless : **no mantiene un estado entre solicitudes de un mismo cliente.**
- > Pero entonces: ¿cómo es posible implementar funcionalidades tales como el login o un carrito de compras ?
- > Se resuelve con **cookies** de dos formas diferentes:
 - Stateful applications
 - Stateless applications.

Stateful applications

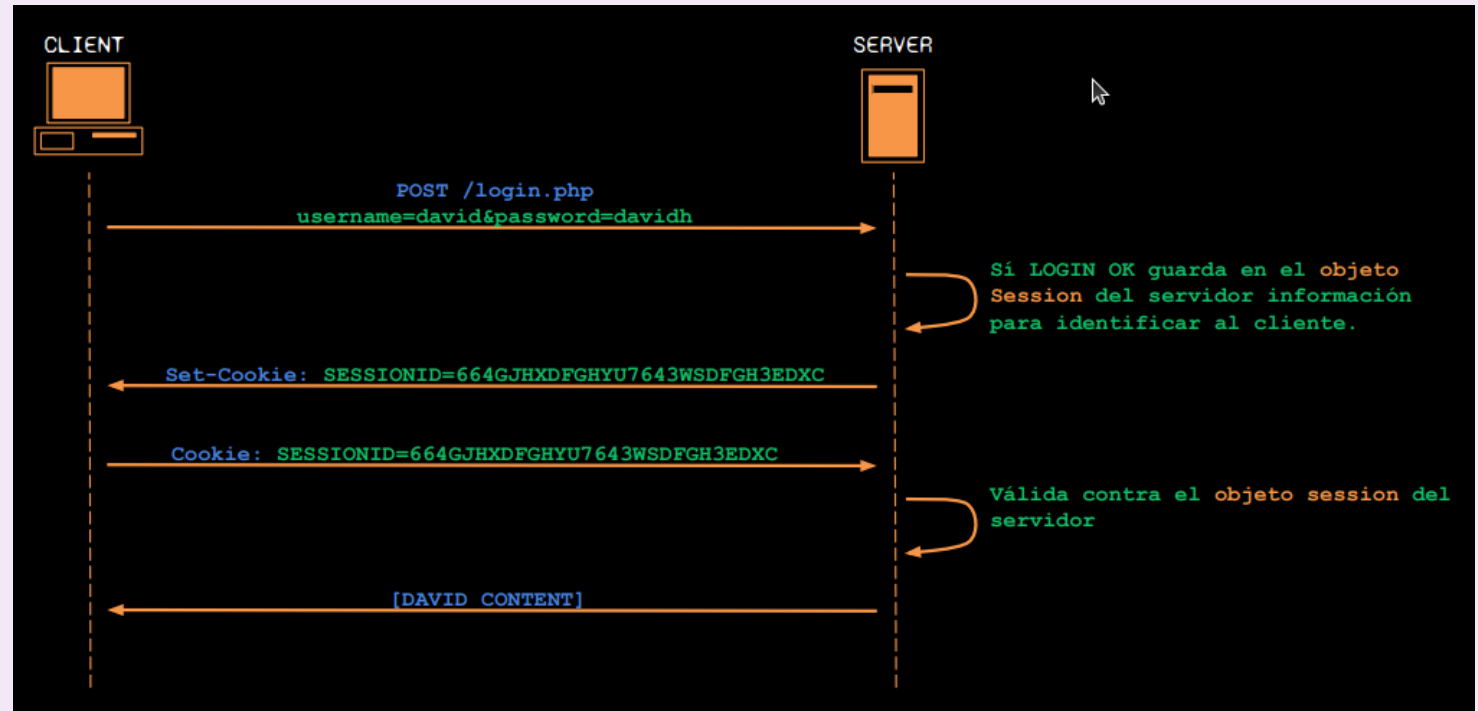
Este enfoque puede tener problemas de escala :

- > La relación cliente servidor existe solo entre el navegador y el servidor específico.

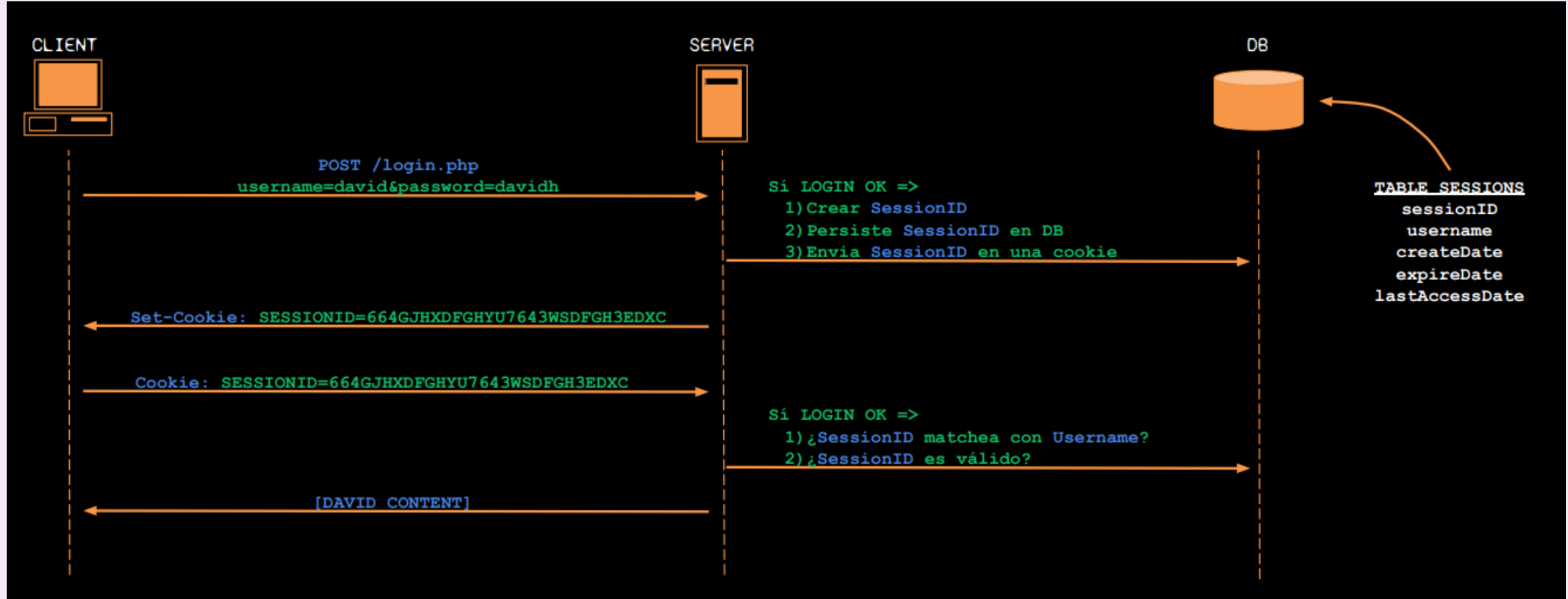
Si algo cambia (cliente o servidor) se rompe .

- > El servidor solo puede manejar tantas conexiones con estado como lo permitan sus recursos informáticos.

- > Si **muchos** clientes establecen sesión con el mismo servidor puede haber problemas de **performance**.



Stateless applications



Dudas?



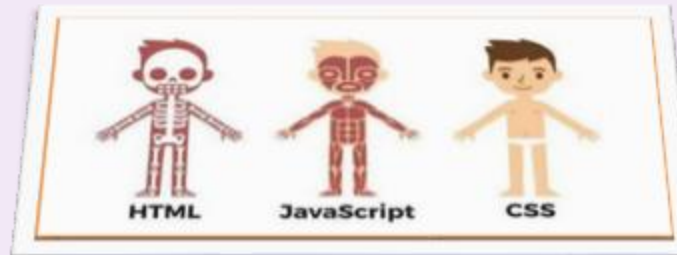
- > Introducción al protocolo HTTP
- > Principales componentes de HTTP
- > Estado y Sesiones HTTP
- > Funcionalidad Web
- > Esquemas de codificación
- > Burp Suite
- > Otras Herramientas

Tecnologías del lado del servidor

- > Las aplicaciones web emplean una amplia gama de tecnologías en el lado del servidor para ofrecer su funcionalidad:
 - Lenguajes de secuencias de comandos como PHP, Java, Python y Perl
 - Plataformas de aplicaciones web como ASP.NET , Spring , Django y Node.js
 - Servidores web como Nginx , Apache , IIS y Tomcat
 - Bases de datos relacionales como MsSQL , Oracle y MySQL ◦ Bases de datos NoSQL tales como MongoDB , Cassandra o Elastic .
 - Otros componentes de back-end como sistemas de archivos, servicios web basados en SOAP, REST y servicios de directorio .

Tecnologías del lado del cliente

- > En general, y a excepción de los servicios web, las aplicaciones del lado del servidor proporcionan una interfaz de usuario del lado del cliente.
- > Esta interfaz suele ser un navegador web capaz de interpretar:
 - HTML
 - CSS
 - JavaScript
- > Además, el navegador implementa el Document Object Model más conocido como DOM.



HyperText Markup Language: Hipervínculos y formularios

```
<a href="?redir=/updates/update29.html">¿Qué está pasando?</a>
```

```
GET /news/8/?redir=/updates/update29.html HTTP/1.1
Host: mdsec.net
...
```



HTML FORM

```
<form action="/secure/login.php?app=quotations" method="post">
  username: <input type="text" name="username"><br>
  password: <input type="password" name="password">
  <input type="hidden" name="redir" value="/secure/home.php">
  <input type="submit" name="submit" value="log in">
</form>
```

Los parámetros están en el cuerpo del mensaje como pares **nombre/valor** igual que los parámetros de URL.

REQUEST

```
POST /secure/login.php?app=quotations HTTP/1.1
Host: wahh-app.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 39
Cookie: SESS=GTnrpx2ss2tSWSnhXJGyG0LJ47MXRsjcFM6Bd
username=daf&password=foo&redir=/secure/home.php&submit=log in
```

La cookie **SESS** que se envió al navegador fue seteada por el servidor en una respuesta anterior a esta.

HyperText Markup Language: Formularios Multipart

multipart/form-data define otra forma de enviar parámetros en formularios.

Una aplicación puede solicitar que los navegadores utilicen esta codificación.

En esta codificación, en el encabezado **Content-Type** se especifica una cadena aleatoria que se utiliza como separador de los parámetros.

```
POST /secure/login.php?app=quotations HTTP/1.1
Host: wahh-app.com
Content-Type: multipart/form-data; boundary=-----7d71385d0a1a
Content-Length: 369
Cookie: SESS=GTnrpx2ss2tSWSnhXJGyG0LJ47MXRsjcFM6Bd

-----7d71385d0a1a
Content-Disposition: form-data; name="username"

daf
-----7d71385d0a1a
Content-Disposition: form-data; name="password"

foo
-----7d71385d0a1a
Content-Disposition: form-data; name="redir"

/secure/home.php
-----7d71385d0a1a
Content-Disposition: form-data; name="submit"

log in
-----7d71385d0a1a
```


Javascript object notation

> JSON es un **formato para expresar objetos en JavaScript**. Se puede utilizar para transferir datos de forma simple. Es procesado de forma nativa por intérpretes de JavaScript.

JSON NOTATION

```
{  
  "name": "Mike Kemp",  
  "id": "8041148671",  
  "email": "fkwitt@layerone.com"  
}
```

REQUEST

```
POST /contacts HTTP/1.0  
Content-Type: application/x-www-form-urlencoded  
Content-Length: 89  
  
Contact={"name":"Mike Kemp","id":"8041148671","email":"pikey@  
clappymonkey.com"}  
&submit=update
```


- > Introducción al protocolo HTTP
- > Principales componentes de HTTP
- > Estado y Sesiones HTTP
- > Funcionalidad Web
- > Esquemas de codificación
- > Burp Suite
- > Otras Herramientas

Qué es un esquema de codificación?

- > El protocolo HTTP y el lenguaje HTML se basan en texto .
- > Por este motivo se desarrollaron diferentes esquemas de codificación para garantizar el manejo seguro de caracteres inusuales y datos binarios .
- > Estos mecanismos son:
 - ASCII
 - UNICODE
 - HTML
 - Base64
 - HEXA

Tabla ASCII

Caracteres ASCII de control			Caracteres ASCII imprimibles			
00	NULL	(carácter nulo)	32	espacio	64	@
01	SOH	(inicio encabezado)	33	!	65	A
02	STX	(inicio texto)	34	"	66	B
03	ETX	(fin de texto)	35	#	67	C
04	EOT	(fin transmisión)	36	\$	68	D
05	ENQ	(consulta)	37	%	69	E
06	ACK	(reconocimiento)	38	&	70	F
07	BEL	(timbre)	39	'	71	G
08	BS	(retroceso)	40	(	72	H
09	HT	(tab horizontal)	41	)	73	I
10	LF	(nueva línea)	42	*	74	J
11	VT	(tab vertical)	43	+	75	K
12	FF	(nueva página)	44	,	76	L
13	CR	(retorno de carro)	45	-	77	M
14	SO	(desplaza afuera)	46	.	78	N
15	SI	(desplaza adentro)	47	/	79	O
16	DLE	(esc.vínculo datos)	48	0	80	P
17	DC1	(control disp. 1)	49	1	81	Q
18	DC2	(control disp. 2)	50	2	82	R
19	DC3	(control disp. 3)	51	3	83	S
20	DC4	(control disp. 4)	52	4	84	T
21	NAK	(conf. negativa)	53	5	85	U
22	SYN	(inactividad sinc)	54	6	86	V
23	ETB	(fin bloque trans)	55	7	87	W
24	CAN	(cancelar)	56	8	88	X
25	EM	(fin del medio)	57	9	89	Y
26	SUB	(sustitución)	58	:	90	Z
27	ESC	(escape)	59	;	91	[
28	FS	(sep. archivos)	60	<	92	\
29	GS	(sep. grupos)	61	=	93	]
30	RS	(sep. registros)	62	>	94	^
31	US	(sep. unidades)	63	?	95	_
127	DEL	(suprimir)				
					96	`
					97	a
					98	b
					99	c
					100	d
					101	e
					102	f
					103	g
					104	h
					105	i
					106	j
					107	k
					108	l
					109	m
					110	n
					111	o
					112	p
					113	q
					114	r
					115	s
					116	t
					117	u
					118	v
					119	w
					120	x
					121	y
					122	z
					123	{
					124	
					125	}
					126	~

ESQUEMAS DE CODIFICACIÓN

Tabla ASCII Extendida

Caracteres ASCII de control				Caracteres ASCII imprimibles				ASCII extendido (Página de código 437)			
00	NULL	(carácter nulo)		32	espacio	64	@	96	`	128	Ç
01	SOH	(inicio encabezado)		33	!	65	A	97	a	129	ü
02	STX	(inicio texto)		34	"	66	B	98	b	130	é
03	ETX	(fin de texto)		35	#	67	C	99	c	131	â
04	EOT	(fin transmisión)		36	\$	68	D	100	d	132	ä
05	ENQ	(consulta)		37	%	69	E	101	e	133	à
06	ACK	(reconocimiento)		38	&	70	F	102	f	134	å
07	BEL	(timbre)		39	'	71	G	103	g	135	ç
08	BS	(retroceso)		40	(	72	H	104	h	136	ê
09	HT	(tab horizontal)		41	)	73	I	105	i	137	ë
10	LF	(nueva línea)		42	*	74	J	106	j	138	è
11	VT	(tab vertical)		43	+	75	K	107	k	139	ï
12	FF	(nueva página)		44	,	76	L	108	l	140	î
13	CR	(retorno de carro)		45	-	77	M	109	m	141	ì
14	SO	(desplaza afuera)		46	.	78	N	110	n	142	Ë
15	SI	(desplaza adentro)		47	/	79	O	111	o	143	Ä
16	DLE	(esc.vínculo datos)		48	0	80	P	112	p	144	É
17	DC1	(control disp. 1)		49	1	81	Q	113	q	145	æ
18	DC2	(control disp. 2)		50	2	82	R	114	r	146	Æ
19	DC3	(control disp. 3)		51	3	83	S	115	s	147	ó
20	DC4	(control disp. 4)		52	4	84	T	116	t	148	ö
21	NAK	(conf. negativa)		53	5	85	U	117	u	149	ò
22	SYN	(inactividad sinc)		54	6	86	V	118	v	150	û
23	ETB	(fin bloque trans)		55	7	87	W	119	w	151	ù
24	CAN	(cancelar)		56	8	88	X	120	x	152	ÿ
25	EM	(fin del medio)		57	9	89	Y	121	y	153	Ö
26	SUB	(sustitución)		58	:	90	Z	122	z	154	Ü
27	ESC	(escape)		59	;	91	[	123	{	155	ø
28	FS	(sep. archivos)		60	<	92	\	124		156	£
29	GS	(sep. grupos)		61	=	93	]	125	}	157	Ø
30	RS	(sep. registros)		62	>	94	^	126	~	158	x
31	US	(sep. unidades)		63	?	95	_			159	f
127	DEL	(suprimir)								160	á
										161	í
										162	ó
										163	ú
										164	ñ
										165	Ñ
										166	ª
										167	º
										168	¿
										169	®
										170	¬
										171	½
										172	¼
										173	¡
										174	«
										175	»
										176	░
										177	▒
										178	▓
										179	▒
										180	┐
										181	Á
										182	Â
										183	Ã
										184	©
										185	ª
										186	»
										187	┐
										188	┐
										189	¢
										190	¥
										191	₱
										192	Ł
										193	ł
										194	┐
										195	┐
										196	—
										197	†
										198	‡
										199	Ä
										200	ℒ
										201	ℒ
										202	ℒ
										203	ℒ
										204	ℒ
										205	≡
										206	≡
										207	≡
										208	ð
										209	Ð
										210	Ê
										211	Ë
										212	È
										213	Ì
										214	Í
										215	Î
										216	Ï
										217	┐
										218	┐
										219	┐
										220	■
										221	┐
										222	┐
										223	■
										224	Ó
										225	Ô
										226	Õ
										227	Ö
										228	ö
										229	Õ
										230	μ
										231	þ
										232	þ
										233	Ú
										234	Û
										235	Ü
										236	ý
										237	Ý
										238	—
										239	’
										240	≡
										241	±
										242	≡
										243	¼
										244	¶
										245	§
										246	÷
										247	.
										248	°
										249	..
										250	·
										251	¹
										252	²
										253	³
										254	■
										255	nbsp

El sistema UNICODE

- > El término Unicode **proviene de los tres objetivos perseguidos: universalidad , uniformidad y unicidad .**
- > Unicode **define tres formas de codificación** bajo el nombre UTF Unicode Transformation Format:
 - **UTF-8:** codificación orientada a byte con símbolos de longitud variable.
 - **UTF-16:** codificación de 16 bits de longitud variable optimizada para la representación del plano básico multilingüe (BMP).
 - **UTF-32:** codificación de 32 bits de longitud fija, y la más sencilla de las tres.

Codificación UNICODE en HTTP

- > Para la transmisión a través de HTTP, la forma codificada en UTF-8 expresa cada byte en hexadecimal precedido por el prefijo %:
 - %c2%a9 → ©
- > La codificación UTF-16 funciona de forma similar a la codificación URL.
- > Se utiliza el prefijo %u seguido del código de carácter Unicode en hexadecimal:
 - %u2215 → /
 - %u00e9 → é

Codificación HTTP

> La codificación HTML se utiliza para representar caracteres problemáticos para que puedan incorporarse de forma segura en un documento HTML.

" → "	< → <
' → '	> → >
& → y	

> Además, cualquier carácter se puede codificar en HTML utilizando su código ASCII en forma decimal o hexadecimal (con x):

" → "	" → "
' → '	' → '

Codificación Base64

- > La codificación Base64 permite representar de forma segura cualquier dato binario utilizando sólo caracteres ASCII imprimibles.

Base64 procesa los datos de entrada en bloques de **tres bytes**.

Cada uno de estos bloques se divide en **cuatro trozos de seis bits** cada uno.

F									O								O							
0	1	0	0	0	1	1	0	0	1	1	0	1	1	1	1	0	1	1	0	1	1	1		
17									38				61				47							
R									m				9				v							

Base64 emplea el siguiente conjunto de caracteres:
 ABCDEFGHIJKLMNOPQRSTUVWXYZabc
 defghijklmnopqrstuvwxyz012345
 6789+/-

Si el último bloque de datos de entrada tiene menos de tres bytes, se rellena con uno o dos caracteres =

¿Por qué Base64?

6 bits de datos permiten 64 posibles permutaciones, por lo que cada fragmento se puede representar mediante un conjunto de 64 caracteres.

Codificación Hexa

- > La codificación hexadecimal **utiliza caracteres ASCII** para representar el bloque de datos binarios a transmitir.
- > Esta codificación también se conoce como **Base16**.
- > Los **datos de entrada se dividen en secuencias de 4 bits** y cada valor (entre 0 y 15 inclusive) se codifica utilizando uno de los 16 símbolos del sistema Hexa.

- > Introducción al protocolo HTTP
- > Principales componentes de HTTP
- > Estado y Sesiones HTTP
- > Funcionalidad Web
- > Esquemas de codificación
- > Burp Suite
- > Otras Herramientas

Que es BurpSuite?

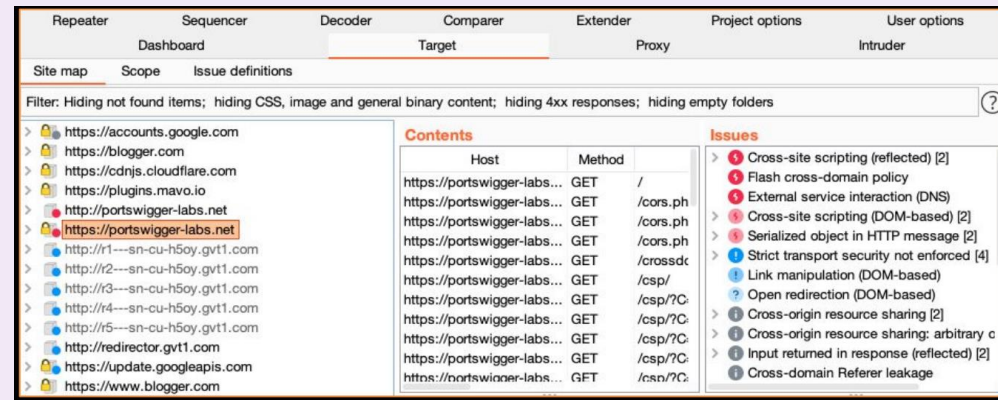
- > Es una colección de distintas herramientas fuertemente integradas con el objetivo de efectuar pruebas de seguridad sobre aplicaciones web modernas.
- > Podemos probar, evaluar y atacar aplicaciones web de forma manual o automatizada de forma simple.
- > Está escrito en lenguaje Java, lo cual lo hace multiplataforma y extremadamente versátil para su uso tanto por principiantes como por profesionales.
- > Está diseñada para ser utilizada con intervención manual.



Principales herramientas

- > Las principales herramientas de Burp son:

- **Scanner**: se utiliza para escanear automáticamente sitios web en busca de contenido y vulnerabilidades de seguridad. Disponible solo con licencia.
- **Intruder**: permite realizar ataques automatizados personalizados.
- **Repeater**: permite modificar manualmente y volver a emitir solicitudes HTTP.
- **Sequencer**: analiza la calidad de la aleatoriedad en los tokens de sesión de una aplicación.
- **Decoder**: codifica y decodifica datos de la aplicación.
- **Comparer**: se utiliza para realizar una comparación visual de bits de datos de la aplicación para encontrar diferencias interesantes.



- > Introducción al protocolo HTTP
- > Principales componentes de HTTP
- > Estado y Sesiones HTTP
- > Funcionalidad Web
- > Esquemas de codificación
- > Burp Suite in a Nutshell
- > Otras Herramientas

DevTools

- > DevTools es un **conjunto de herramientas para desarrolladores** integrado en el navegador Chrome (por ejemplo)
- > Podemos **interactuar con aplicaciones web usando la consola**; ver encabezados, vistas previas y respuestas; y analizar los archivos fuente de la aplicación web.

Panel	Función
Elements	Permite ver el CSS de la página actual y el DOM, lo que le permite inspeccionar el HTML que construye la página web.
Console	Proporciona alertas y permite interactuar con el depurador de JavaScript para modificar la página web actual.
Sources	Contiene los directorios que componen la aplicación web y el contenido de los archivos fuente.
Network	Enumera todas las solicitudes de archivos de origen que conforman la perspectiva del cliente de la aplicación web.
Performance	Proporciona una forma de registrar y analizar todos los eventos que tienen lugar al cargar una página web.
Memory	Permite registrar y analizar cómo interactúa el navegador con la memoria de su sistema.
Application	Proporciona el manifiesto de la aplicación, elementos de almacenamiento (como cookies e información de sesión), caché y servicios en segundo plano.
Security	Proporciona información sobre el cifrado de tránsito, los orígenes del contenido de origen y los detalles del certificado.

Dudas?

